North Bengal International University

Lab Report: 04

Exp Name : Longest common Subsequence(LCS)

Subject : Analysis of algorithm Sessional

Subject Code : CSE 0612-2212

Submitted By:

Name : Md. Tanvir Rahman Pranto
Dept : CSE(26)
Roll : 0712340012101005
Semester : 3rd Year, 5th semester.

Submitted To:

Name : Afsana Tasnim
Designation : (HOD)Lecturer
Dept : CSE
Date :

Task No: 01

Task Name: Longest common subsequence.

Introduction: Given two strings, L1 and L2, the task is to find the length of the Longest Common Subsequence. If there is no common subsequence**,** return 0. A subsequence is a string generated from the original string by deleting 0 or more characters, without changing the relative order of the remaining characters.

For example, subsequences of "ABC" are "", "A", "B", "C", "AB", "AC", "BC" and "ABC". In general, a string of length n has $2n$ subsequence.

Algorithm:

1. **Read two strings** from the user, call them `X` and `Y`.
2. **Find their lengths** `m` and `n`.
3. **Create a table** `L[m+1][n+1]` and set all values to `0`.
4. **Fill the table**:
   - For each character in `X` (i = 1 to m):
     - For each character in `Y` (j = 1 to n):
       - If `X[i-1] == Y[j-1]`, then `L[i][j] = L[i-1][j-1] + 1`
       - Otherwise, `L[i][j] = max(L[i-1][j], L[i][j-1])`
5. **The LCS length** is `L[m][n]`.
6. **Find the LCS string** by starting from `L[m][n]` and moving backwards:
   - If characters match, add it to the LCS string and move diagonally up-left.
   - If not, move to the cell with the larger value (up or left).
7. **Print the LCS length** and the LCS string.

code:

```
#include <stdio.h>

#define MAX 100

int main() {
    char X[MAX], Y[MAX], lcs[MAX];
    int L[MAX][MAX];
    int m = 0, n = 0;
    int i, j;
```

```c
    printf("Enter first string: ");
    scanf("%s", X);
    printf("Enter second string: ");
    scanf("%s", Y);

    /* Find lengths of strings */
    for (m = 0; X[m] != '\0'; m++);
    for (n = 0; Y[n] != '\0'; n++);

    /* Initialize table with 0 */
    for (i = 0; i <= m; i++)
        for (j = 0; j <= n; j++)
            L[i][j] = 0;

    /* Fill table */
    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++) {
            if (X[i-1] == Y[j-1])
                L[i][j] = L[i-1][j-1] + 1;
            else
               L[i][j] = (L[i-1][j] > L[i][j-1]) ? L[i-1][j] : L[i][j-1];
        }
    }

    /* Reconstruct LCS */
    int index = L[m][n];
    lcs[index] = '\0';
    i = m; j = n;
    while (i > 0 && j > 0) {
        if (X[i-1] == Y[j-1]) {
            lcs[index-1] = X[i-1];
            i--; j--; index--;
        } else if (L[i-1][j] > L[i][j-1])
            i--;
        else
            j--;
    }

    printf("Length of LCS: %d\n", L[m][n]);
    printf("LCS: %s\n", lcs);

    return 0;
}
```

output:

```
Enter first string: ABCBDAB
Enter second string: BDCAB
Length of LCS: 4
LCS: BCAB
```

Conclusion: The LCS problem is a clear example of dynamic programming: build a table of subproblem results and use it to derive both the LCS length and the subsequence itself. The DP approach runs in O(m * n) time for strings of lengths m and n, and uses O(m * n) space for reconstruction. This lab demonstrated building the DP table and backtracking to reconstruct the sequence, and included a plain C implementation without user-defined functions.